САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

# Отчет по лабораторной работе №3
# по курсу «Проектирование и реализация баз данных»
# Вариант 14 “**Служба заказа такси**”

Выполнил:
Лукошников Д.В.
К3242

Санкт-Петербург
2026 г.

**Цель работы:** овладеть практическими навыками создания таблиц базы данных PostgreSQL 1X, заполнения их рабочими данными, резервного копирования и восстановления БД.

**Оборудование:** компьютерный класс.

**Программное обеспечение:** СУБД PostgreSQL 1X, pgAdmin 4.

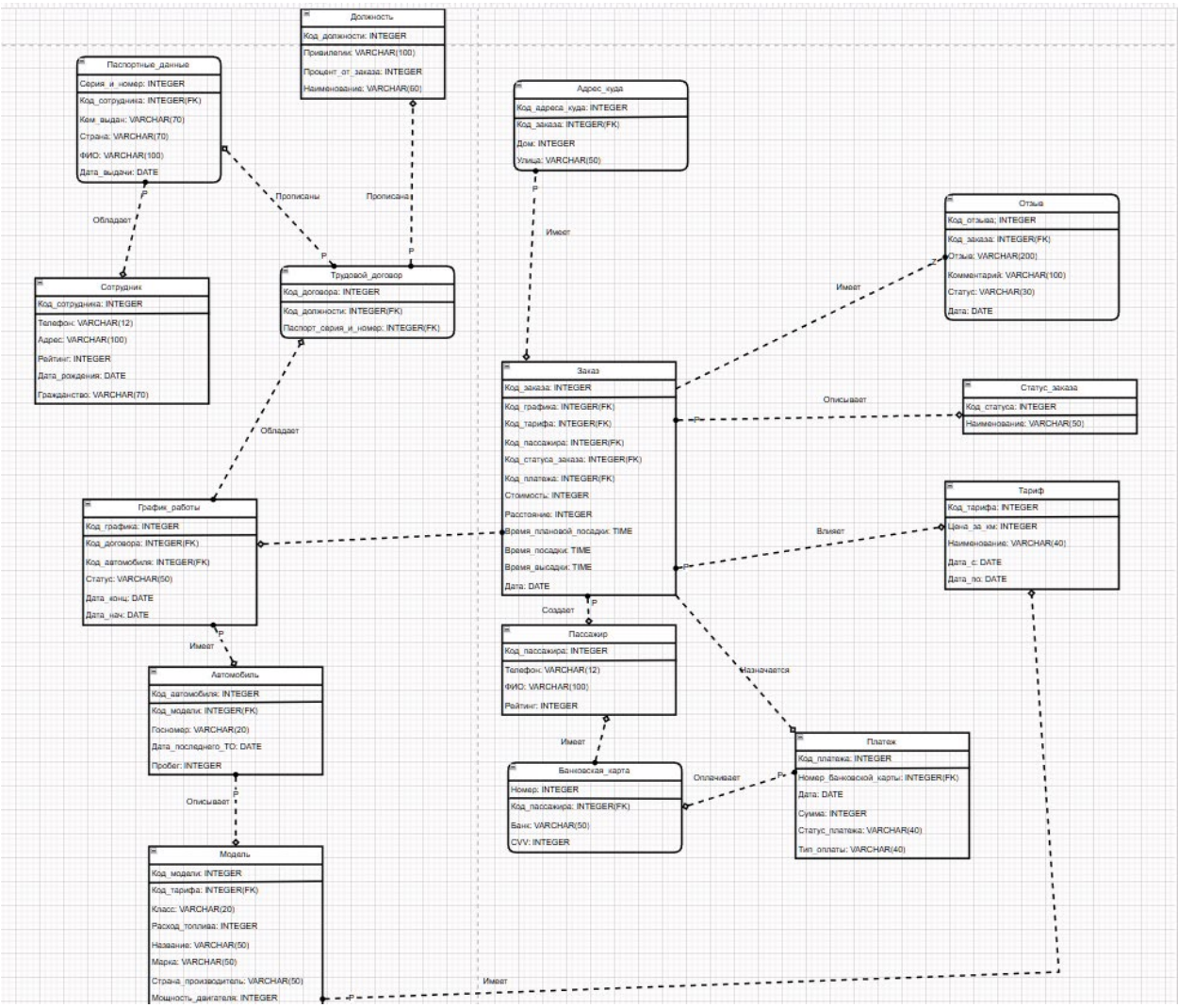
**Практическое задание:**

1. Создать базу данных с использованием pgAdmin 4 (согласно индивидуальному заданию).
2. Создать схему в составе базы данных.
3. Создать таблицы базы данных.
4. Установить ограничения на данные: *Primary Key, Unique, Check, Foreign Key*.
5. Заполнить таблицы БД рабочими данными.
6. Создать резервную копию БД.
7. Восстановить БД.

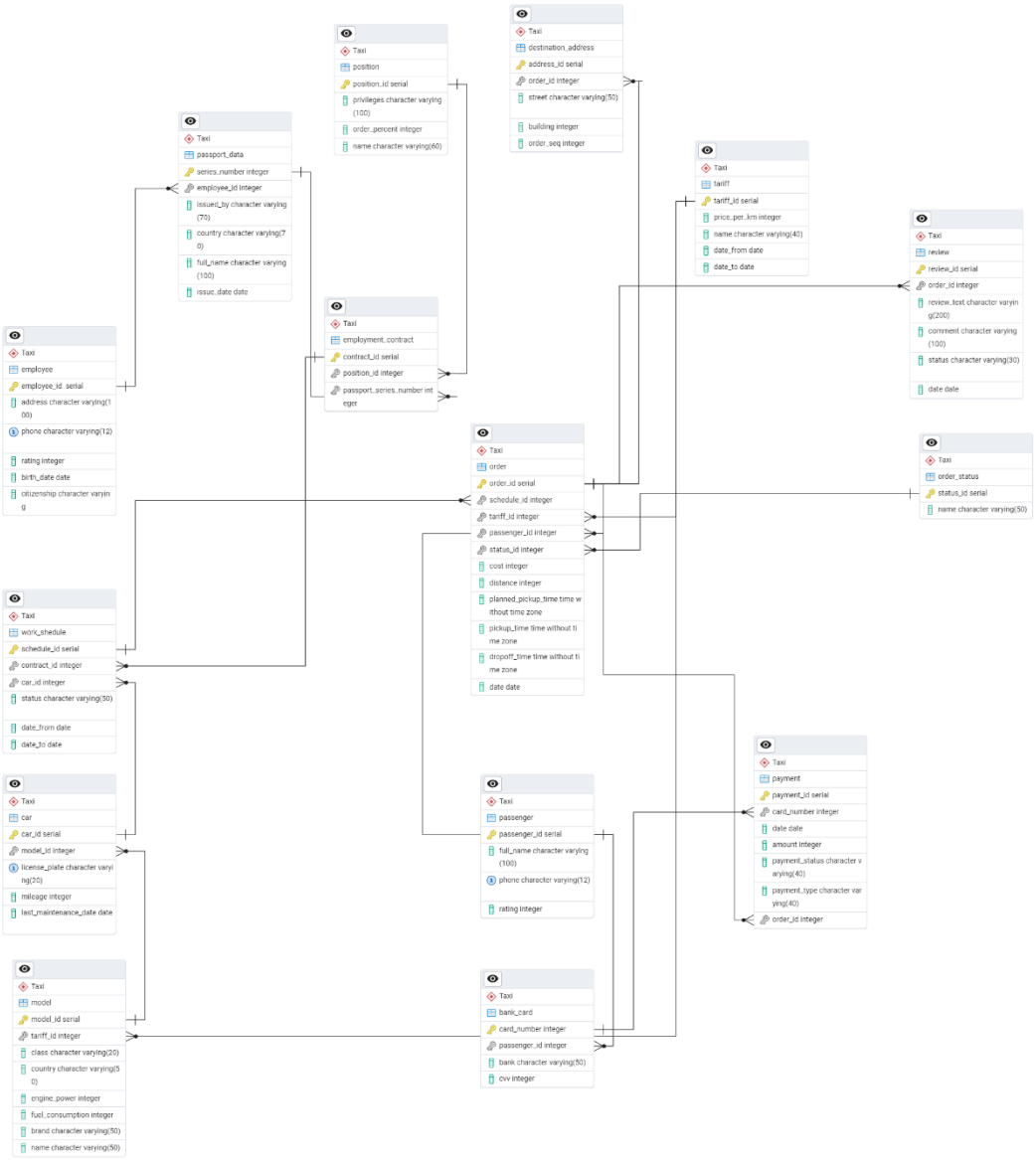
# Выполнение

## Вариант 14. БД «Служба заказа такси»

## Схема из ЛР2



## Схема логической модели базы данных, сгенерированная в Generate ERD




## Dump, содержащий скрипты работы с БД.

### Создание БД

```
CREATE DATABASE "TaxiOrder" WITH TEMPLATE = template0 ENCODING = 'UTF8'
LOCALE_PROVIDER = libc LOCALE = 'ru-RU';
```

### Создание схемы

```
CREATE SCHEMA "Taxi";
```

## Создание таблиц и ограничения на них

```
CREATE TABLE "Taxi".bank_card
(
    card_number integer NOT NULL,
    passenger_id integer NOT NULL,
    bank character varying(50) NOT NULL,
```

```
    cvv integer NOT NULL,
    year_to integer,
    month_to integer,
    CONSTRAINT bank_card_pkey PRIMARY KEY (card_number),
    CONSTRAINT fk_card_passenger FOREIGN KEY (passenger_id)
    REFERENCES "Taxi".passenger (passenger_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT card_cvv_check CHECK (cvv >= 100 AND cvv <= 999),
    CONSTRAINT card_number_check CHECK (card_number > 0),
    CONSTRAINT month_check CHECK (month_to >= 1 AND month_to <= 12)
NOT VALID,
    CONSTRAINT year_check CHECK (year_to > 0 AND year_to < 100) NOT
VALID
)

CREATE TABLE "Taxi".car
(
    car_id serial NOT NULL,
    model_id integer NOT NULL,
    license_plate character varying(20) NOT NULL,
    mileage integer NOT NULL,
    last_maintenance_date date,
    CONSTRAINT car_pkey PRIMARY KEY (car_id),
    CONSTRAINT car_license_plate_unique UNIQUE (license_plate),
    CONSTRAINT fk_car_model FOREIGN KEY (model_id)
        REFERENCES "Taxi".model (model_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT car_mileage_check CHECK (mileage >= 0)
)

CREATE TABLE "Taxi".destination_address
(
    address_id serial NOT NULL,
    order_id integer NOT NULL,
    street character varying(50) NOT NULL,
    building integer NOT NULL,
    order_seq integer NOT NULL,
    CONSTRAINT destination_address_pkey PRIMARY KEY (address_id),
    CONSTRAINT fk_address_order FOREIGN KEY (order_id)
        REFERENCES "Taxi"."order" (order_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT address_building_check CHECK (building > 0),
```

```
    CONSTRAINT seq_check CHECK (order_seq >= 1 AND order_seq < 100)
NOT VALID
)

CREATE TABLE "Taxi".employee
(
    employee_id serial NOT NULL,
    address character varying(100) NOT NULL,
    phone character varying(12) NOT NULL,
    rating integer NOT NULL,
    birth_date date NOT NULL,
    citizenship character varying NOT NULL,
    CONSTRAINT employee_pkey PRIMARY KEY (employee_id),
    CONSTRAINT employee_phone_unique UNIQUE (phone),
    CONSTRAINT employee_phone_check CHECK (phone::text ~ '^\+7[0-
9]{10}$'::text),
    CONSTRAINT employee_rating_check CHECK (rating >= 0 AND rating <=
5),
    CONSTRAINT employee_birth_date_check CHECK (birth_date > '1900-01-
01'::date AND birth_date <= CURRENT_DATE)
)

CREATE TABLE "Taxi".employment_contract
(
    contract_id serial NOT NULL,
    position_id integer NOT NULL,
    passport_series_number integer NOT NULL,
    CONSTRAINT employment_contract_pkey PRIMARY KEY (contract_id),
    CONSTRAINT fk_contract_passport FOREIGN KEY
(passport_series_number)
        REFERENCES "Taxi".passport_data (series_number) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
        NOT VALID,
    CONSTRAINT fk_contract_position FOREIGN KEY (position_id)
        REFERENCES "Taxi"."position" (position_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
        NOT VALID
)

CREATE TABLE IF NOT EXISTS "Taxi".model
(
    model_id serial NOT NULL,
    tariff_id integer NOT NULL,
```

```
    class character varying(20) NOT NULL,
    country character varying(50) NOT NULL,
    engine_power integer NOT NULL,
    fuel_consumption integer NOT NULL,
    brand character varying(50) NOT NULL,
    name character varying(50) NOT NULL,
    CONSTRAINT model_pkey PRIMARY KEY (model_id),
    CONSTRAINT fk_model_tariff FOREIGN KEY (tariff_id)
        REFERENCES "Taxi".tariff (tariff_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT model_class_check CHECK (class::text = ANY
(ARRAY['эконом'::character varying, 'бизнес'::character varying,
'комфорт'::character varying, 'грузовой'::character
varying]::text[])),
    CONSTRAINT model_engine_power_check CHECK (engine_power > 0),
    CONSTRAINT model_fuel_consumption_check CHECK (fuel_consumption >=
0)
)

CREATE TABLE "Taxi"."order"
(
    order_id serial NOT NULL DEFAULT,
    schedule_id integer NOT NULL,
    tariff_id integer NOT NULL,
    passenger_id integer NOT NULL,
    status_id integer NOT NULL,
    cost integer NOT NULL,
    distance integer NOT NULL,
    planned_pickup_time time without time zone NOT NULL,
    pickup_time time without time zone,
    dropoff_time time without time zone,
    date date NOT NULL,
    CONSTRAINT order_pkey PRIMARY KEY (order_id),
    CONSTRAINT fk_order_passenger FOREIGN KEY (passenger_id)
        REFERENCES "Taxi".passenger (passenger_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_order_schedule FOREIGN KEY (schedule_id)
        REFERENCES "Taxi".work_shedule (schedule_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_order_status FOREIGN KEY (status_id)
        REFERENCES "Taxi".order_status (status_id) MATCH SIMPLE
        ON UPDATE NO ACTION
```

```
        ON DELETE NO ACTION,
    CONSTRAINT fk_order_tariff FOREIGN KEY (tariff_id)
        REFERENCES "Taxi".tariff (tariff_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT order_distance_check CHECK (distance > 0),
    CONSTRAINT order_date_check CHECK (date <= CURRENT_DATE),
    CONSTRAINT order_times_check CHECK ((pickup_time IS NULL OR
pickup_time >= planned_pickup_time) AND (dropoff_time IS NULL OR
dropoff_time >= pickup_time))
)

CREATE TABLE IF NOT EXISTS "Taxi".order_status
(
    status_id serial NOT NULL,
    name character varying(50) NOT NULL,
    CONSTRAINT order_status_pkey PRIMARY KEY (status_id)
)

CREATE TABLE "Taxi".passenger
(
    passenger_id serial NOT NULL,
    full_name character varying(100) NOT NULL,
    phone character varying(12) NOT NULL,
    rating integer NOT NULL,
    CONSTRAINT passenger_pkey PRIMARY KEY (passenger_id),
    CONSTRAINT passenger_phone_unique UNIQUE (phone),
    CONSTRAINT passenger_phone_check CHECK (phone::text ~ '^\+7[0-
9]{10}$'::text),
    CONSTRAINT passenger_rating_check CHECK (rating > 0 AND rating <=
5) NOT VALID
)

CREATE TABLE IF NOT EXISTS "Taxi".passport_data
(
    series_number integer NOT NULL,
    employee_id integer NOT NULL,
    issued_by character varying(70) NOT NULL,
    country character varying(70) NOT NULL,
    full_name character varying(100) NOT NULL,
    issue_date date NOT NULL,
    CONSTRAINT passport_data_pkey PRIMARY KEY (series_number),
    CONSTRAINT fk_passport_employee FOREIGN KEY (employee_id)
        REFERENCES "Taxi".employee (employee_id) MATCH SIMPLE
        ON UPDATE NO ACTION
```

```
        ON DELETE CASCADE,
    CONSTRAINT passport_issue_date_check CHECK (issue_date > '1900-01-
01'::date AND issue_date <= CURRENT_DATE)
)

CREATE TABLE "Taxi".payment
(
    payment_id serial NOT NULL,
    card_number integer,
    date date NOT NULL,
    amount integer NOT NULL,
    payment_status character varying(40) NOT NULL,
    payment_type character varying(40) NOT NULL,
    order_id integer,
    CONSTRAINT payment_pkey PRIMARY KEY (payment_id),
    CONSTRAINT fk_payment FOREIGN KEY (order_id)
        REFERENCES "Taxi"."order" (order_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
        NOT VALID,
    CONSTRAINT fk_payment_card FOREIGN KEY (card_number)
        REFERENCES "Taxi".bank_card (card_number) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT payment_date_check CHECK (date > '1900-01-01'::date AND
date <= CURRENT_DATE),
    CONSTRAINT payment_amount_check CHECK (amount > 0),
    CONSTRAINT payment_status_check CHECK (payment_status::text = ANY
(ARRAY['не оплачен'::character varying, 'в процессе'::character
varying, 'оплачен'::character varying]::text[])),
    CONSTRAINT payment_type_check CHECK (payment_type::text = ANY
(ARRAY['картой'::character varying, 'наличкой'::character
varying]::text[]))
)

CREATE TABLE "Taxi"."position"
(
    position_id serial NOT NULL,
    privileges character varying(100),
    order_percent integer NOT NULL,
    name character varying(60) NOT NULL,
    CONSTRAINT position_pkey PRIMARY KEY (position_id),
    CONSTRAINT position_percent_check CHECK (order_percent >= 1 AND
order_percent <= 100),
```

```
    CONSTRAINT position_name_check CHECK (name::text = ANY
(ARRAY['водитель'::character varying, 'администратор'::character
varying, 'грузчик'::character varying]::text[]))
)

CREATE TABLE IF NOT EXISTS "Taxi".review
(
    review_id serial NOT NULL,
    order_id integer NOT NULL,
    review_text character varying(200),
    comment character varying(100),
    status character varying(30) NOT NULL,
    date date NOT NULL,
    CONSTRAINT review_pkey PRIMARY KEY (review_id),
    CONSTRAINT fk_review_order FOREIGN KEY (order_id)
        REFERENCES "Taxi"."order" (order_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT review_status_check CHECK (status::text = ANY
(ARRAY['опубликован'::character varying, 'отклонен'::character
varying, 'в процессе'::character varying]::text[])),
    CONSTRAINT review_date_check CHECK (date > '1900-01-01'::date AND
date <= CURRENT_DATE)
)

CREATE TABLE "Taxi".tariff
(
    tariff_id serial NOT NULL,
    price_per_km integer NOT NULL,
    name character varying(40) NOT NULL,
    date_from date NOT NULL,
    date_to date,
    CONSTRAINT tariff_pkey PRIMARY KEY (tariff_id),
    CONSTRAINT tariff_price_check CHECK (price_per_km > 0),
    CONSTRAINT tariff_dates_check CHECK (date_from > '1900-01-
01'::date AND (date_to IS NULL OR date_to >= date_from))
)

CREATE TABLE "Taxi".work_shedule
(
    schedule_id serial NOT NULL,
    contract_id integer NOT NULL,
    car_id integer,
    status character varying(50) NOT NULL,
    date_from date NOT NULL,
```

```
    date_to date,
    CONSTRAINT work_shedule_pkey PRIMARY KEY (schedule_id),
    CONSTRAINT fk_schedule_car FOREIGN KEY (car_id)
        REFERENCES "Taxi".car (car_id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT fk_schedule_contract FOREIGN KEY (contract_id)
        REFERENCES "Taxi".employment_contract (contract_id) MATCH
SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT schedule_status_check CHECK (status::text = ANY
(ARRAY['не начат'::character varying, 'в процессе'::character varying,
'закончен'::character varying]::text[])),
    CONSTRAINT schedule_dates_check CHECK (date_from > '1900-01-
01'::date AND (date_to IS NULL OR date_to >= date_from))
)
```

## Комментарии по созданию таблиц и ограничений:

- Таблицы создавались с помощью ключевого слова CREATE TABLE
- Каждый атрибут имеет свое название, тип данных и возможное ограничение
- Если значение поля не должно быть не определенным, то указываем NOT NULL
- Если столбец является первичным ключем отношения, то добавляем CONSTRAINT со словом PRIMARY KEY
- Если поле таблицы ссылается на поле из другой таблицы, то то добавляем CONSTRAINT со словом FOREIGN KEY и REFERENCES на таблицу на которую мы ссылаемся

## Вставка проверочных данных с помощью insert

```
INSERT INTO "Taxi".employee (employee_id, address, phone, rating,
birth_date, citizenship) VALUES
(1, 'ул. Ленина, д. 10, кв. 5', '+79031234567', 5, '1985-03-15',
'Россия'),
(2, 'пр. Мира, д. 25, кв. 12', '+79037654321', 4, '1990-07-22',
'Россия'),
(3, 'ул. Гагарина, д. 7, кв. 3', '+79039876543', 5, '1982-11-30',
'Россия'),
(4, 'ул. Пушкина, д. 15, кв. 8', '+79031112233', 3, '1995-05-18',
'Россия'),
(5, 'пр. Победы, д. 3, кв. 21', '+79034445566', 4, '1988-09-10',
'Россия'),
(6, 'ул. Советская, д. 45, кв. 7', '+79037778899', 5, '1979-12-25',
'Россия'),
(7, 'ул. Кирова, д. 12, кв. 4', '+79039990011', 4, '1992-04-03',
'Россия'),
```

```
(8, 'пр. Ленинградский, д. 8, кв. 16', '+79036667788', 5, '1983-08-19', 'Россия');

INSERT INTO "Taxi".passport_data (series_number, employee_id, issued_by, country, full_name, issue_date) VALUES
(4501123456, 1, 'УФМС России по г. Москва', 'Россия', 'Сергеев Сергей Петрович', '2010-05-20'),
(4502123456, 2, 'УФМС России по г. Санкт-Петербург', 'Россия', 'Петрова Анна Ивановна', '2012-08-15'),
(4503123456, 3, 'УФМС России по г. Екатеринбург', 'Россия', 'Иванов Дмитрий Алексеевич', '2008-03-10'),
(4504123456, 4, 'УФМС России по г. Казань', 'Россия', 'Смирнова Елена Викторовна', '2015-11-25'),
(4505123456, 5, 'УФМС России по г. Нижний Новгород', 'Россия', 'Кузнецов Андрей Сергеевич', '2011-07-30'),
(4506123456, 6, 'УФМС России по г. Новосибирск', 'Россия', 'Попова Ольга Дмитриевна', '2005-12-05'),
(4507123456, 7, 'УФМС России по г. Самара', 'Россия', 'Соколов Михаил Павлович', '2013-09-12'),
(4508123456, 8, 'УФМС России по г. Челябинск', 'Россия', 'Лебедева Татьяна Николаевна', '2014-04-18');

INSERT INTO "Taxi"."position" (position_id, privileges, order_percent, name) VALUES
(1, 'Право управления автомобилями категории B', 10, 'водитель'),
(2, 'Доступ к администрированию системы, управление персоналом', 5, 'администратор'),
(3, 'Право погрузочно-разгрузочных работ', 8, 'грузчик'),
(4, 'Право управления грузовыми автомобилями категории C', 12, 'водитель'),
(5, 'Старший смены, контроль работы', 7, 'администратор'),
(6, 'Работа с особо тяжелыми грузами', 9, 'грузчик');

INSERT INTO "Taxi".employment_contract (contract_id, employee_id, position_id, passport_series_number, start_date, end_date) VALUES
(1, 1, 1, 4501123456, '2020-01-15', NULL),
(2, 2, 2, 4502123456, '2021-03-10', NULL),
(3, 3, 1, 4503123456, '2019-11-20', NULL),
(4, 4, 3, 4504123456, '2022-02-05', NULL),
(5, 5, 4, 4505123456, '2021-08-30', '2024-08-30'),
(6, 6, 5, 4506123456, '2018-05-12', NULL),
(7, 7, 3, 4507123456, '2023-01-25', NULL),
(8, 8, 1, 4508123456, '2022-09-15', NULL);
```

```
INSERT INTO "Taxi".tariff (tariff_id, price_per_km, name, date_from,
date_to) VALUES
(1, 30, 'Эконом', '2024-01-01', NULL),
(2, 50, 'Комфорт', '2024-01-01', NULL),
(3, 80, 'Бизнес', '2024-01-01', NULL),
(4, 40, 'Детский', '2024-01-01', NULL),
(5, 100, 'Грузовой', '2024-01-01', NULL),
(6, 35, 'Эконом (ночной)', '2024-01-01', NULL),
(7, 70, 'Комфорт+', '2024-01-01', '2024-12-31');

INSERT INTO "Taxi".model (model_id, tariff_id, class, country,
engine_power, fuel_consumption, brand, name) VALUES
(1, 1, 'эконом', 'Россия', 106, 7, 'Lada', 'Vesta'),
(2, 1, 'эконом', 'Россия', 87, 6, 'Lada', 'Granta'),
(3, 2, 'комфорт', 'Южная Корея', 150, 8, 'Hyundai', 'Solaris'),
(4, 2, 'комфорт', 'Южная Корея', 128, 7, 'Kia', 'Rio'),
(5, 3, 'бизнес', 'Германия', 190, 9, 'Mercedes-Benz', 'E-Class'),
(6, 3, 'бизнес', 'Германия', 249, 10, 'BMW', '5 Series'),
(7, 4, 'грузовой', 'Россия', 150, 12, 'ГАЗ', 'Газель Next'),
(8, 4, 'грузовой', 'Россия', 215, 15, 'КамАЗ', '4308'),
(9, 5, 'эконом', 'Франция', 110, 6, 'Renault', 'Logan'),
(10, 2, 'комфорт', 'Япония', 140, 7, 'Toyota', 'Corolla'),
(11, 1, 'эконом', 'Китай', 100, 6, 'Haval', 'Jolion');

INSERT INTO "Taxi".car (car_id, model_id, license_plate, mileage,
last_maintenance_date) VALUES
(1, 1, 'А123БВ777', 45000, '2024-05-15'),
(2, 2, 'Б456ГД777', 67800, '2024-06-20'),
(3, 3, 'В789ЕЖ777', 23400, '2024-07-10'),
(4, 4, 'Г0123И777', 89200, '2024-04-05'),
(5, 5, 'Д345КЛ777', 56700, '2024-08-25'),
(6, 6, 'Е678МН777', 12300, '2024-09-01'),
(7, 7, 'Ж901ОП777', 112300, '2024-03-12'),
(8, 8, 'З234РС777', 34500, '2024-07-30'),
(9, 9, 'И567ТУ777', 67800, '2024-06-18'),
(10, 10, 'Й890ФХ777', 23400, '2024-08-05'),
(11, 1, 'К123ЦЧ777', 56700, '2024-05-22'),
(12, 3, 'Л456ШЩ777', 89000, '2024-04-10');

INSERT INTO "Taxi".work_shedule (schedule_id, contract_id, car_id,
status, date_from, date_to) VALUES
(1, 1, 1, 'в процессе', '2024-10-01', NULL),
(2, 2, NULL, 'в процессе', '2024-10-01', NULL),
(3, 3, 2, 'в процессе', '2024-10-01', NULL),
(4, 4, 7, 'закончен', '2024-09-01', '2024-09-30'),
```

```
(5, 4, 7, 'в процессе', '2024-10-01', NULL),
(6, 5, 8, 'не начат', '2024-11-01', NULL),
(7, 6, NULL, 'в процессе', '2024-10-01', NULL),
(8, 7, 4, 'закончен', '2024-09-15', '2024-09-30'),
(9, 7, 5, 'в процессе', '2024-10-01', NULL),
(10, 8, 3, 'в процессе', '2024-10-01', NULL),
(11, 1, 6, 'не начат', '2024-10-15', NULL),
(12, 2, 9, 'закончен', '2024-09-10', '2024-09-20');

INSERT INTO "Taxi".passenger (passenger_id, full_name, phone, rating)
VALUES
(1, 'Иванов Иван Иванович', '+79161112233', 4),
(2, 'Петрова Мария Сергеевна', '+79162223344', 5),
(3, 'Сидоров Алексей Петрович', '+79163334455', 3),
(4, 'Козлова Елена Владимировна', '+79164445566', 4),
(5, 'Михайлов Дмитрий Андреевич', '+79165556677', 5),
(6, 'Федорова Ольга Николаевна', '+79166667788', 4),
(7, 'Николаев Павел Сергеевич', '+79167778899', 3),
(8, 'Алексеева Анна Викторовна', '+79168889900', 5),
(9, 'Степанов Артем Игоревич', '+79169990011', 4),
(10, 'Дмитриева Наталья Павловна', '+79160001122', 4),
(11, 'Андреев Максим Романович', '+79169991122', 4),
(12, 'Романов Виктор Викторович', '+79162227788', 5);

INSERT INTO "Taxi".bank_card (card_number, passenger_id, bank, cvv,
year_to, month_to) VALUES
(12345678, 1, 'Сбербанк', 123, NULL, NULL),
(23456789, 2, 'Тинькофф', 456, NULL, NULL),
(34567890, 3, 'ВТБ', 789, NULL, NULL),
(45678901, 4, 'Альфа-Банк', 234, NULL, NULL),
(56789012, 5, 'Сбербанк', 567, NULL, NULL),
(67890123, 6, 'Тинькофф', 890, NULL, NULL),
(78901234, 7, 'ВТБ', 345, NULL, NULL),
(89012345, 8, 'Газпромбанк', 678, NULL, NULL),
(90123456, 9, 'Райффайзенбанк', 901, NULL, NULL),
(11223344, 10, 'Сбербанк', 112, NULL, NULL),
(22334455, 11, 'Тинькофф', 223, NULL, NULL),
(33445566, 12, 'ВТБ', 334, NULL, NULL);
INSERT INTO "Taxi".payment (payment_id, card_number, date, amount,
payment_status, payment_type, order_id) VALUES
(1, 12345678, '2024-10-01', 450, 'оплачен', 'картой', NULL),
(2, 23456789, '2024-10-01', 780, 'оплачен', 'картой', NULL),
(3, NULL, '2024-10-02', 320, 'оплачен', 'наличкой', NULL),
(4, 34567890, '2024-10-02', 560, 'оплачен', 'картой', NULL),
(5, 45678901, '2024-10-03', 890, 'оплачен', 'картой', NULL),
```

```
(6, NULL, '2024-10-03', 430, 'оплачен', 'наличкой', NULL),
(7, 56789012, '2024-10-04', 1240, 'в процессе', 'картой', NULL),
(8, 67890123, '2024-10-04', 670, 'оплачен', 'картой', NULL),
(9, 78901234, '2024-10-05', 350, 'не оплачен', 'картой', NULL),
(10, NULL, '2024-10-05', 520, 'оплачен', 'наличкой', NULL),
(11, 89012345, '2024-10-06', 980, 'оплачен', 'картой', NULL),
(12, 90123456, '2024-10-06', 430, 'оплачен', 'картой', NULL),
(13, 11223344, '2024-10-07', 670, 'оплачен', 'картой', NULL),
(14, 22334455, '2024-10-07', 890, 'оплачен', 'картой', NULL);

INSERT INTO "Taxi".order_status (status_id, name) VALUES
(1, 'не закончен'),
(2, 'в процессе'),
(3, 'закончен');

INSERT INTO "Taxi"."order" (order_id, schedule_id, tariff_id,
passenger_id, status_id, payment_id, cost, distance,
planned_pickup_time, pickup_time, dropoff_time, date) VALUES
(1, 1, 1, 1, 3, 1, 450, 15, '10:00:00', '10:05:00', '10:35:00', '2024-
10-01'),
(2, 2, 2, 2, 3, 2, 780, 16, '12:30:00', '12:35:00', '13:15:00', '2024-
10-01'),
(3, 3, 1, 3, 3, 3, 320, 11, '14:00:00', '14:02:00', '14:30:00', '2024-
10-02'),
(4, 4, 2, 4, 3, 4, 560, 12, '15:45:00', '15:50:00', '16:25:00', '2024-
10-02'),
(5, 5, 3, 5, 3, 5, 890, 12, '09:15:00', '09:18:00', '09:55:00', '2024-
10-03'),
(6, 6, 1, 6, 3, 6, 430, 15, '11:30:00', '11:35:00', '12:10:00', '2024-
10-03'),
(7, 7, 1, 7, 2, 7, 1240, 16, '17:00:00', '17:10:00', NULL, '2024-10-
04'),
(8, 8, 2, 8, 3, 8, 670, 14, '18:30:00', '18:32:00', '19:15:00', '2024-
10-04'),
(9, 9, 3, 9, 1, 9, 350, 5, '20:00:00', NULL, NULL, '2024-10-05'),
(10, 10, 4, 10, 3, 10, 520, 6, '08:45:00', '08:47:00', '09:20:00',
'2024-10-05'),
(11, 11, 2, 11, 3, 11, 980, 21, '13:15:00', '13:20:00', '14:10:00',
'2024-10-06'),
(12, 12, 1, 12, 3, 12, 430, 15, '16:45:00', '16:48:00', '17:25:00',
'2024-10-06'),
(13, 11, 2, 10, 3, 13, 670, 14, '11:00:00', '11:02:00', '11:45:00',
'2024-10-07'),
(14, 9, 3, 1, 3, 14, 890, 12, '15:30:00', '15:35:00', '16:10:00',
'2024-10-07');
```

```
INSERT INTO "Taxi".review (review_id, order_id, review_text, comment,
status, date) VALUES
(1, 1, 'Отличная поездка, вежливый водитель', 'Всё понравилось',
'опубликован', '2024-10-01'),
(2, 2, 'Машина чистая, приехали вовремя', 'Рекомендую', 'опубликован',
'2024-10-01'),
(3, 3, 'Хорошо, но немного опоздали', 'Могли бы и побыстрее',
'опубликован', '2024-10-02'),
(4, 4, NULL, 'Без комментариев', 'отклонен', '2024-10-02'),
(5, 5, 'Прекрасный сервис, буду пользоваться еще', 'Спасибо!',
'опубликован', '2024-10-03'),
(6, 6, 'Нормально', NULL, 'опубликован', '2024-10-03'),
(7, 7, NULL, NULL, 'в процессе', '2024-10-04'),
(8, 8, 'Водитель профессионал, машина комфортная', NULL,
'опубликован', '2024-10-04'),
(9, 9, NULL, NULL, 'отклонен', '2024-10-05'),
(10, 10, 'Быстро и недорого', 'Цена/качество супер', 'опубликован',
'2024-10-05'),
(11, 11, 'Отличная поездка', NULL, 'опубликован', '2024-10-06'),
(12, 12, 'Всё хорошо', 'Спасибо', 'опубликован', '2024-10-06'),
(13, 13, 'Отлично', NULL, 'опубликован', '2024-10-07'),
(14, 14, 'Хороший водитель', 'Понравилось', 'опубликован', '2024-10-
07');

INSERT INTO "Taxi".destination_address (address_id, order_id, street,
building, order_seq) VALUES
(1, 1, 'ул. Тверская', 15, NULL),
(2, 2, 'пр. Ленинский', 25, NULL),
(3, 3, 'ул. Арбат', 7, NULL),
(4, 4, 'пл. Красная', 1, NULL),
(5, 5, 'пр. Невский', 42, NULL),
(6, 6, 'ул. Мясницкая', 13, NULL),
(7, 7, 'ул. Пятницкая', 8, NULL),
(8, 8, 'пр. Мира', 32, NULL),
(9, 9, 'ул. Новый Арбат', 21, NULL),
(10, 10, 'наб. Москва-реки', 5, NULL),
(11, 11, 'ул. Воздвиженка', 9, NULL),
(12, 12, 'ул. Ильинка', 4, NULL),
(13, 13, 'ул. Большая Дмитровка', 12, NULL),
(14, 14, 'ул. Малая Бронная', 18, NULL);
```

## Выводы:

В ходе выполнения лабораторной работы была создана база данных "TaxiOrder" в СУБД PostgreSQL 17 с использованием графического интерфейса pgAdmin 4, а также разработана схема "Taxi" для организации данных таксопарка. Было создано 15 таблиц, соответствующих предметной области, с установленными ограничениями первичных и внешних ключей, проверочными ограничениями CHECK и уникальности UNIQUE для обеспечения целостности данных. Таблицы были заполнены тестовыми данными с помощью INSERT-запросов, после чего созданы две резервные копии базы данных в форматах CUSTOM (для восстановления) и PLAIN (для получения SQL-листинга), а также выполнено успешное восстановление базы данных из резервной копии, что подтвердило работоспособность механизмов резервного копирования.